10e. SIMD: Contiguous Access and Unit Strides

Martin Alfaro
PhD in Economics

INTRODUCTION

This section examines performance differences between copies and views in the context of SIMD instructions. While views may initially appear advantageous due to their ability to **avoid memory allocations**, this section demonstrates that copies possess a memory layout better aligned with SIMD requirements.

To understand why this occurs, let's first revisit how SIMD improves performance in the first place. SIMD accelerates computation by applying the same operation simultaneously to multiple data elements. This is made possible by specialized vector registers in the CPU, which can hold several values at once (e.g., 4 floating-point numbers or 4 integers). With data packed into these registers, operations like additions or multiplications can complete with a single instruction instead of many.

For SIMD to fully leverage this vector-based processing, **data must adhere to specific rules regarding memory organization**. Specifically, there are two fundamental conditions.

- **Contiguous Memory Layout**: data elements must reside in adjacent memory addresses, one after another, with no gaps between them. Freshly allocated arrays automatically meet this requirement, enabling entire segments to be loaded directly into vector registers. In contrast, array views don't guarantee contiguity. By referencing the original data structure, views could result in highly irregular memory access patterns depending on the elements selected.
- **Unit Strides**: strides refer to the step size between consecutive memory accesses. Unit strides mean in particular that elements are accessed sequentially. For example, iterating through a freshly allocated vector `x` using `eachindex(x)` ensures unit stride: each access moves to the next adjacent address in memory. This contrasts with non-unit strides, such as accessing every other element with `1:2:length(x)`, or unpredictable accesses patterns resulting from an index list like `[1, 5, 3, 4]`.

Because SIMD performs best when both conditions hold, choosing between views and copies involves trade-offs. On one hand, using views reduces memory allocations, but at the expense of making SIMD less effective if views violate contiguity or unit stride. On the other hand, copies create contiguous arrays amenable to SIMD, but at the cost of increased memory usage. This section examines how these trade-offs play out in practice.

REVIEW OF INDEXING APPROACHES

The performance trade-offs between copies and views are highly dependent on the slicing method employed. For this reason, we begin with a brief overview of common slicing techniques: vector indexing, ranges, and Boolean indexing. We illustrate each below.

VECTOR INDEXING

```
x        = [10, 20, 30]

indices  = [3, 2, 1]
elements = x[indices]
```

```
julia> indices
3-element Vector{Int64}:
 3
 2
 1
julia> elements
3-element Vector{Int64}:
 30
 20
 10
```

RANGES

```
x        = [2, 3, 4, 5, 6]

indices_1 = 1:length(x)         # unit strides, equivalent to 1:1:length(x)
indices_2 = 1:2:length(x)       # strides two
```

```
julia> collect(indices_1)
5-element Vector{Int64}:
 1
 2
 3
 4
 5
julia> collect(indices_2)
3-element Vector{Int64}:
 1
 3
 5
```

BOOLEAN INDEXING

```
x        = [20, 10, 30]

indices  = x .> 15
elements = x[indices]
```

```
julia> indices
3-element BitVector:
 1
 0
 1
julia> elements
2-element Vector{Int64}:
 20
 30
```

Vector Indexing selects elements by explicitly listing their indices. This approach offers the greatest flexibility in choosing arbitrary elements. However, it can also result in memory access patterns that are highly non-contiguous.

Ranges are a structured form of indexing that selects elements according to a given stride. Their general syntax is `<first index>:<stride>:<last index>`, where omitting `<stride>` defaults to a step size of 1. Ranges with unit stride maintain contiguous access patterns, while larger strides produce predictable but non-contiguous access patterns.

Finally, **Boolean indexing** uses a Boolean vector with `true` entries indicating the elements to retain. This technique is commonly used when broadcasting conditions to filter data. Because the resulting pattern of selected elements often lacks regularity, Boolean indexing typically leads to irregular and fragmented memory access.

BENEFITS OF SEQUENTIAL ACCESS

In a previous section, we highlighted the benefits of using views over copies when working with slices: by maintaining references to the original data, views avoid memory allocation. Yet, we also remarked that copies could outperform views in some scenarios. We're now in a position to explain in more depth why this occurs.

Creating a copy of some data structure involves allocating the information in a new contiguous block of memory. This layout ensures that elements sit next to each other, thus offering two key advantages: faster element retrieval and a more effective use of SIMD instructions. Views can't guarantee this property. If the referenced elements are scattered throughout memory, views will necessarily introduce irregular access patterns.

An analogy may help clarify this distinction. Imagine retrieving books from a library. If every book you need is neatly arranged on a single shelf, collecting them is straightforward: you move once, grab each, and proceed. This mirrors contiguous memory access. Conversely, if the books are dispersed

across different floors and sections, each retrieval demands additional time and effort. This is akin to non-contiguous access.

The analogy extends further by adding a cart that lets you carry several books at once. The cart plays the role of vector registers. They enable SIMD instructions to process multiple adjacent data elements in a single operation.

ILLUSTRATING EACH BENEFIT IN ISOLATION

Next, we illustrate the potential advantages of using copies over views. All examples are based on a reduction operation.

To isolate the effects of copying itself, we create the slices outside the benchmarked function. In this way, performance differences aren't influenced by the memory allocations associated with copying.

We begin with a case in which the elements accessed follow a regular pattern, although not with unit stride.

VIEW (NO SIMD)

```
x = rand(5_000_000)
y = @view x[1:2:length(x)]

function foo(y)
    output = 0.0

    for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($y)
  1.540 ms (0 allocations: 0 bytes)
```

VIEW (SIMD)

```
x = rand(5_000_000)
y = @view x[1:2:length(x)]

function foo(y)
    output = 0.0

    @simd for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($y)
  1.831 ms (0 allocations: 0 bytes)
```

COPY (NO SIMD)

```
x = rand(5_000_000)
y = x[1:2:length(x)]

function foo(y)
    output = 0.0

    for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($y)
  1.094 ms (0 allocations: 0 bytes)
```

COPY (SIMD)

```
x = rand(5_000_000)
y = x[1:2:length(x)]

function foo(y)
    output = 0.0

    @simd for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($y)
  644.124 μs (0 allocations: 0 bytes)
```

The examples make three points. First, when comparing the two view-based variants, non-unit strides prevent the compiler from applying SIMD efficiently. As a result, the extra work required to rearrange elements for vectorization ends up slowing down the computation.

Second, when comparing the implementations without SIMD, the copy performs better. Since neither version benefits from SIMD, the speedup stems entirely from contiguous memory access.

Finally, when the computation is based on a copy and applies SIMD instructions, the implementation combines the benefits of contiguous memory access and vectorization. This yields the fastest execution among all variants.

The benefits of copying become more evident when access pattern is unpredictable. To illustrate, consider accessing elements of a vector `x` via the indices returned by `sortperm`. This function returns the indices that would sort the vector. For instance, given `x = [5, 4, 6]`, `sort(x)` returns

`[4, 5, 6]`, while `sortperm(x)` provides `[2, 1, 3]`. Assuming that `x` hasn't been ordered beforehand, views that access elements according to the indices of `sortperm(x)` will result in unpredictable memory access pattern.

VIEW (NO SIMD)

```
x       = rand(5_000_000)

indices = sortperm(x)
y       = @view x[indices]

function foo(y)
    output = 0.0

    for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($y)
  29.691 ms (0 allocations: 0 bytes)
```

VIEW (SIMD)

```
x       = rand(5_000_000)

indices = sortperm(x)
y       = @view x[indices]

function foo(y)
    output = 0.0

    @simd for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($y)
  31.315 ms (0 allocations: 0 bytes)
```

COPY (NO SIMD)

```
x       = rand(5_000_000)

indices = sortperm(x)
y       = x[indices]

function foo(y)
    output = 0.0

    for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($y)
  2.276 ms (0 allocations: 0 bytes)
```

COPY (SIMD)

```
x       = rand(5_000_000)

indices = sortperm(x)
y       = x[indices]

function foo(y)
    output = 0.0

    @simd for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($y)
  1.634 ms (0 allocations: 0 bytes)
```

A similar conclusion can be obtained when slices are created with Boolean indexing.

VIEW (NO SIMD)

```
x       = rand(5_000_000)

indices = x .> 0.5
y       = @view x[indices]

function foo(y)
    output = 0.0

    for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($y)
  2.465 ms (0 allocations: 0 bytes)
```

VIEW (SIMD)

```
x       = rand(5_000_000)

indices = x .> 0.5
y       = @view x[indices]

function foo(y)
    output = 0.0

    @simd for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($y)
  2.326 ms (0 allocations: 0 bytes)
```

COPY (NO SIMD)

```
x       = rand(5_000_000)

indices = x .> 0.5
y       = x[indices]

function foo(y)
    output = 0.0

    for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($y)
  1.095 ms (0 allocations: 0 bytes)
```

COPY (SIMD)

```
x       = rand(5_000_000)

indices = x .> 0.5
y       = x[indices]

function foo(y)
    output = 0.0

    @simd for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($y)
  622.719 µs (0 allocations: 0 bytes)
```

COPIES VS VIEWS: TOTAL EFFECTS

The previous examples defined slices outside the benchmarked functions. In this way, we avoided the memory allocations associated with copying. The decision was deliberate, with the goal of isolating the benefits of sequential memory access and SIMD.

In practice, however, the choice between copies and views must ultimately account for the cost of memory allocations. Once this overhead is incorporated, the existence of trade-off is revealed. Consequently, whether copies or views yield better performance is highly case-specific. For this

reason, benchmarking becomes the only reliable way to determine which approach delivers superior performance in a given context.

In what follows, we present cases that account for all these effects simultaneously. We begin with an example where views prove more performant. This scenario is characterized by significant memory allocation costs, but only marginal benefits from SIMD due to the simplicity of the operation being executed.

VIEW

```
x       = rand(5_000_000)
indices = sortperm(x)

function foo(x, indices)
    y      = @view x[indices]
    output = 0.0

    @simd for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($x, $indices)
  33.449 ms (0 allocations: 0 bytes)
```

COPY

```
x       = rand(5_000_000)
indices = sortperm(x)

function foo(x, indices)
    y      = x[indices]
    output = 0.0

    @simd for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($x, $indices)
  49.233 ms (2 allocations: 38.147 MiB)
```

By contrast, the next operation is more computationally intensive, therefore markedly benefiting from SIMD instructions. This explains why a copy outperforms a view.

VIEW

```
x       = rand(5_000_000)
indices = sortperm(x)

function foo(x, indices)
    y      = @view x[indices]
    output = 0.0

    @simd for a in y
        output += a^(3/2)
    end

    return output
end
```

```
julia> @btime foo($x, $indices)
  407.670 ms (0 allocations: 0 bytes)
```

COPY

```
x       = rand(5_000_000)
indices = sortperm(x)

function foo(x, indices)
    y      = x[indices]
    output = 0.0

    @simd for a in y
        output += a^(3/2)
    end

    return output
end
```

```
julia> @btime foo($x, $indices)
  159.115 ms (2 allocations: 38.147 MiB)
```

CHOOSING WITHOUT BENCHMARKING

Certain access patterns make it possible to anticipate the faster approach, without the need of benchmarking.

One scenario where views typically outperform copies is when elements are referenced sequentially. In those cases, views provide the same benefits as copies, but without incurring memory allocations.

VIEW (NO SIMD)

```
x       = rand(1_000_000)
indices = 1:length(x)

function foo(x, indices)
    y      = @view x[indices]
    output = 0.0

    for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($x, $indices)
  481.124 µs (0 allocations: 0 bytes)
```

VIEW (WITH SIMD)

```
x       = rand(1_000_000)
indices = 1:length(x)

function foo(x, indices)
    y      = @view x[indices]
    output = 0.0

    @simd for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($x, $indices)
  121.409 µs (0 allocations: 0 bytes)
```

COPY (WITH SIMD)

```
x       = rand(1_000_000)
indices = 1:length(x)

function foo(x, indices)
    y      = x[indices]
    output = 0.0

    @simd for a in y
        output += a
    end

    return output
end
```

```
julia> @btime foo($x, $indices)
  677.714 μs (2 allocations: 7.629 MiB)
```

Conversely, a common scenario where copies tend to outperform views is when multiple operations are executed on the same slice. In cases like this, the cost of creating the copy is often amortized over several computations. As a result, the speed gains from both contiguous memory access and SIMD instructions usually outweight the initial allocation overhead.

VIEW (NO SIMD)

```
x       = rand(5_000_000)
indices = sortperm(x)

function foo(x, indices)
    y      = @view x[indices]
    output1, output2, output3 = (0.0 for _ in 1:3)

    for a in y
        output1 += a^(3/2)
        output2 += a / 3
        output3 += a * 2.5
    end

    return output1, output2, output3
end
```

```
julia> @btime foo($x, $indices)
  404.078 ms (0 allocations: 0 bytes)
```

VIEW (WITH SIMD)

```
x       = rand(5_000_000)
indices = sortperm(x)

function foo(x, indices)
    y      = @view x[indices]
    output1, output2, output3 = (0.0 for _ in 1:3)

    @simd for a in y
        output1 += a^(3/2)
        output2 += a / 3
        output3 += a * 2.5
    end

    return output1, output2, output3
end
```

```
julia> @btime foo($x, $indices)
  422.823 ms (0 allocations: 0 bytes)
```

COPY (WITH SIMD)

```
x       = rand(5_000_000)
indices = sortperm(x)

function foo(x, indices)
    y      = x[indices]
    output1, output2, output3 = (0.0 for _ in 1:3)

    @simd for a in y
        output1 += a^(3/2)
        output2 += a / 3
        output3 += a * 2.5
    end

    return output1, output2, output3
end
```

```
julia> @btime foo($x, $indices)
  155.528 ms (2 allocations: 38.147 MiB)
```